

CTG GO — Arquitectura y Stack Tecnológico — MVP

Versión 0.1 — Decisión técnica inicial

Objetivo: definir una base tecnológica simple, mantenible y escalable para construir el MVP, alineada con la experiencia actual del responsable técnico y preparada para incorporar especialistas más adelante.

1. Principios de arquitectura

- Priorizar tecnologías ampliamente utilizadas, con documentación madura y buen soporte por herramientas de IA.
- Aprovechar Python y SQL como núcleo del desarrollo, reduciendo la cantidad de tecnologías nuevas.
- Comenzar con un monolito modular: una sola aplicación backend desplegable, dividida internamente por dominios y responsabilidades.
- Separar claramente reglas de negocio, interpretación mediante IA, persistencia, procesos asíncronos e integraciones externas.
- Mantener la interfaz web como canal principal del MVP. WhatsApp, audio, OCR y API externa quedan como extensiones posteriores.
- Toda operación con efectos oficiales en ARCA requiere confirmación humana explícita y trazabilidad.

2. Arquitectura recomendada

La arquitectura inicial será un monolito modular con frontend separado. Esto permite desarrollar rápido sin convertir el sistema en una masa indivisible ni asumir desde el comienzo el costo operativo de microservicios.

2.1 Componentes

- Frontend web: aplicación React que consume la API del backend.
- API backend: FastAPI, responsable de autenticación, permisos, reglas de negocio, validaciones, operaciones CRUD y coordinación del flujo.
- Base transaccional: PostgreSQL como fuente principal de verdad.
- Worker asíncrono: Celery para trabajos durables, reintentos e integraciones que no deben depender de una solicitud HTTP abierta.
- Broker de mensajes: RabbitMQ, tecnología ya conocida por el responsable técnico.
- Integración IA: adaptador interno desacoplado del proveedor para interpretar texto y devolver datos estructurados; la IA no escribe directamente en ARCA ni decide reglas oficiales.
- Integración ARCA/SISA: adaptadores específicos, aislados del dominio y con registro completo de solicitudes, respuestas, errores e identificadores de correlación.

- Proxy y TLS: Nginx delante del frontend y la API.

2.2 Flujo técnico principal

Operador → Gestión → texto libre o carga estructurada → interpretación IA → BorradorCPE estructurado → validaciones determinísticas → revisión y confirmación humana → trabajo asíncrono → ARCA → actualización y seguimiento independiente de cada CPE. La Gestión y cada CPE conservarán estados independientes. Una emisión múltiple generará un trabajo por CPE: el éxito o error de una no revertirá las demás.

3. Stack tecnológico propuesto

3.1 Backend — Decisión recomendada

- Python 3.13 como versión inicial, fijada por archivo de proyecto y contenedor.
- FastAPI para API REST y documentación OpenAPI automática.
- Pydantic v2 para contratos, validación y estructuras intercambiadas con la IA.
- SQLAlchemy 2.0 para acceso a datos y Alembic para migraciones.
- psycopg 3 como driver PostgreSQL.
- uv para dependencias, entorno y archivo de bloqueo reproducible.
- pytest, pytest-asyncio y Testcontainers para pruebas.

Criterio: el backend debe organizarse por módulos de dominio —usuarios, clientes, datos maestros, gestiones, CPE, asistente, ARCA, SISA y auditoría— evitando una carpeta genérica de servicios donde termine viviendo todo.

3.2 Base de datos — Decisión recomendada

- PostgreSQL como base relacional principal. Es software libre; el costo real será el hosting, almacenamiento, copias y operación.
- JSONB sólo para payloads variables, respuestas externas y evidencia técnica. Los datos centrales de negocio deben modelarse en columnas y relaciones.
- UUID para identificadores internos; CUIT, dominio y demás identificadores oficiales con restricciones únicas según corresponda.
- Migraciones exclusivamente mediante Alembic; ningún cambio manual no documentado en producción.
- Auditoría append-only para acciones sensibles, actor, fecha, motivo, resultado y correlación externa.
- Outbox transaccional para publicar trabajos sin perder consistencia entre base y cola.

3.3 Frontend — Decisión recomendada

- React + TypeScript + Vite. Es una combinación estándar, documentada y apropiada para un frontend que será generado y mantenido inicialmente con asistencia de IA.
- Tailwind CSS y shadcn/ui para construir una interfaz consistente con componentes cuyo código permanece dentro del proyecto.

- TanStack Query para estado del servidor, caché y sincronización.
- React Hook Form + Zod para formularios y validación de entrada.
- Cliente TypeScript generado desde el contrato OpenAPI de FastAPI para evitar que frontend y backend inventen contratos diferentes.
- Playwright para pruebas end-to-end de los flujos críticos.

No recomiendo Next.js para el MVP: CTG GO es una aplicación operativa autenticada, no un sitio orientado a SEO. Vite reduce capas y evita mantener un segundo backend accidental dentro del frontend.

3.4 Procesos asíncronos — Decisión recomendada

- Celery + RabbitMQ para emisión, consulta de estados, sincronización SISA, reintentos controlados, generación de documentos y futuras tareas OCR o mensajería.
- Cada tarea externa deberá ser idempotente, tener timeout, política de reintentos, clave de correlación y registro del resultado.
- No usar FastAPI BackgroundTasks para operaciones críticas: no ofrece durabilidad ante reinicios.

3.5 Archivos y documentos

- No guardar PDFs, imágenes o audios dentro de PostgreSQL.
- Usar almacenamiento compatible con S3; MinIO en desarrollo y un servicio S3-compatible administrado en producción.
- Guardar en PostgreSQL únicamente metadatos, ubicación, hash, tipo, tamaño, propietario y relación con la Gestión o CPE.

4. Docker y despliegue

Docker es una decisión correcta. La recomendación para el MVP es Docker Compose sobre un único servidor virtual, con servicios separados para frontend, API, worker, RabbitMQ, PostgreSQL y Nginx. PostgreSQL podría migrarse luego a un servicio administrado sin cambiar la aplicación.

- Un Dockerfile por componente, imágenes pequeñas, usuario no root y healthchecks.
- Configuración por variables de entorno; secretos fuera del repositorio.
- Ambientes separados: local, pruebas/staging y producción.
- CI/CD para ejecutar lint, pruebas, construir imágenes, aplicar migraciones de forma controlada y desplegar.
- No adoptar Kubernetes en el MVP. Docker Compose es suficiente mientras exista un servidor y una escala acotada.

5. Seguridad, trazabilidad y resiliencia

- Autenticación con sesiones seguras en cookies HttpOnly; contraseñas con Argon2id y RBAC básico: administrador y operador.

- Cifrado TLS; credenciales de ARCA, tokens y claves de IA gestionados como secretos, nunca en base de datos en texto plano ni en logs.
- Registro inmutable de confirmaciones humanas y acciones oficiales.
- Sanitización de logs para no exponer CUIT, credenciales, tokens ni payloads completos sensibles.
- Backups automáticos de PostgreSQL, cifrados y almacenados fuera del servidor; prueba periódica de restauración.
- Rate limiting, protección CSRF cuando corresponda, control de CORS y límites de tamaño de archivos.

6. Observabilidad

- Logs JSON estructurados con correlation_id por solicitud, Gestión, CPE y trabajo asíncrono.
- Métricas técnicas y de negocio: tiempos, errores, reintentos, tareas en cola, emisiones exitosas/fallidas y diferencias de estado.
- Prometheus + Grafana para métricas y alertas; Loki para logs cuando el volumen lo justifique.
- Healthchecks y alertas mínimas desde el primer despliegue.

7. Estructura de repositorio

Se recomienda un monorepo para el MVP:

- backend/: API, dominio, adaptadores y migraciones.
- frontend/: aplicación React.
- deploy/: Compose, Nginx y scripts de despliegue.
- docs/: decisiones de arquitectura, contratos e integración.
- tests/: pruebas de integración y end-to-end compartidas cuando corresponda.

El monorepo facilita cambios coordinados y trabajo asistido por IA. No implica mezclar responsabilidades: cada componente mantiene límites, pruebas y dependencias propios.

8. Reglas específicas para el uso de IA

- La salida del modelo debe validarse contra esquemas Pydantic antes de persistirse.
- Las reglas legales, fiscales y operativas son código determinístico y versionado; no quedan escondidas dentro de prompts.
- Los prompts y versiones de modelos deben registrarse para reproducir resultados relevantes.
- La IA sólo propone o completa borradores. La confirmación humana es obligatoria para emitir, anular, desviar, confirmar, rechazar o ejecutar otra acción oficial.
- Implementar un puerto de proveedor de IA para poder cambiar de modelo sin reescribir el dominio.

9. Decisiones que quedan abiertas

- Proveedor y modalidad de hosting: VPS autoadministrado versus servicios administrados.
- Estrategia definitiva de autenticación: implementación local inicial versus proveedor OIDC cuando existan clientes externos.
- Servicio de almacenamiento S3-compatible para producción.
- Proveedor o modelos de IA, presupuesto, retención de datos y condiciones de privacidad.
- Servicios oficiales y mecanismos exactos disponibles para ARCA, SISA, delegaciones, descarga de documentos y estados posteriores.
- Repositorio y plataforma CI/CD definitiva.

10. Evolución prevista

Fase MVP: monolito modular, interfaz web, texto libre, PostgreSQL, FastAPI, React, Celery/RabbitMQ y Docker Compose.

Fase de consolidación: mejorar observabilidad, seguridad, automatización de despliegue, backups y eventualmente utilizar PostgreSQL administrado.

Fase de crecimiento: separar servicios sólo cuando métricas, equipos o necesidades de escalado lo justifiquen; evaluar Kubernetes recién en ese contexto.

11. Decisión ejecutiva

Se aprueba como propuesta técnica inicial: Python + FastAPI, PostgreSQL, React + TypeScript + Vite, Celery + RabbitMQ, Nginx y Docker Compose, implementados como monorepo y monolito modular con frontend separado.

Esta selección maximiza el aprovechamiento de la experiencia actual, mantiene costos y complejidad bajo control, ofrece abundante documentación y permite que el frontend sea asistido por IA sin comprometer los contratos ni las reglas centrales del producto.

12. Fuentes técnicas de referencia

FastAPI — despliegue en contenedores

PostgreSQL — documentación oficial

Docker Compose — documentación oficial

Vite — guía oficial

Celery con RabbitMQ — documentación oficial

SQLAlchemy 2.0 — documentación oficial